

카테고리 없음

[RAG] 논문 리뷰

yuha933 2025. 6. 30. 12:26

1. Introduction

RAG의 등장 배경

기존 모델 (ex. GPT, BERT, T5, BART 등)

- **사전학습된 신경 언어 모델** (외부 문서나 지식베이스를 실시간으로 검색하지 않아도, 학습된 파라미터 속에 간접적으로 저장된 지식을 기반으로 답변을 생성하는 모델)
- 문제
 - 쉽게 **확장**하거나 **수정**할 수 없음
 - 예측 결과에 대한 명확한 **근거** 제공 어려움
 - **사실이 아닌 내용**을 생성할 수 있음

부분적 해결책 (ex. REALM, ORQA)

- **파라메트릭 메모리**(모델의 파라미터 안에 저장된 지식)와 **논파라메트릭 메모리** (외부 문서를 검색해서 활용하는 지식)를 결합한 하이브리드 모델
- 대표 모델 : REALM, ORQA (마스킹 언어 모델 : 파라메트릭 모델 + 미분 가능한 검색기 : 논파라메트릭 모델)
- 특징
 - 지식 **수정, 확장 가능** (외부 문서를 바꾸면 모델을 다시 학습하지 않아도 지식을 업데이트할 수 있음)
 - 검색된 지식 **점검, 해석 가능** (답변에 사용된 문서를 즉, 출처를 보여줄 수 있음 -> 사람이 확인 가능)
- 문제
 - 개방형 도메인에서의 추출 기반 질문 응답 extractive QA(정답이 되는 문장이나 구절을 그대로 "**찾아서**" 보여주는 방식)에만 적용 -> 언어모델이 가진 문장 생성, 추론 능력을 거의 쓰지 못했으며, 다양한 자연어 처리 과제에 활용할 수 없음

해결책 (RAG)

- **RAG** : 하이브리드 구조를 NLP의 핵심 모델인 Seq2seq구조에 도입
- 문제 해결
 - 기존 Seq2seq 모델의 문제 : 지식이 파라미터에 고정되어 있음 (수정, 출처 제공, 최신 정보 반영 불가)
 - 기존 하이브리드 모델의 문제 : 언어 모델의 문장 생성, 추론 능력 활용 부족
- RAG는 두 방식의 **단점을 서로 보완**하는 구조가 됨

RAG의 구조 및 작동 원리

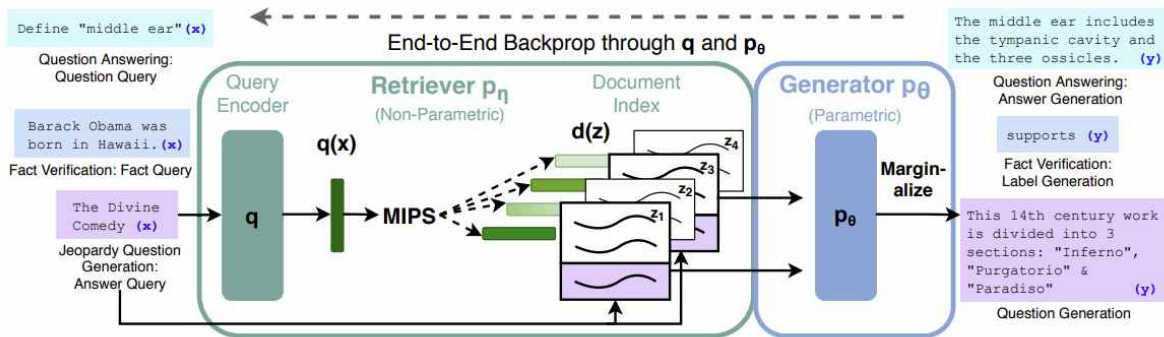


Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i . For final prediction y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

RAG 구조

RAG : 사전학습된 **생성 모델에 검색 기능을 결합**하고, 검색과 생성을 함께 파인튜닝하여, 외부 문서를 실시간으로 참고해 더 정확하고 근거 있는 답변을 생성할 수 있게 만든 모델

• 구성 요소

- 파라메트릭 모델 : 사전학습된 seq2seq 트랜스포머
- 논파라메트릭 메모리 : 위키피디아 문서의 밀집 벡터 인덱스

• 작동 원리

1. 사전학습된 신경망 기반 검색기(DPR)를 통해 문서 접근 (논파라메트릭 메모리) : 벡터 유사도 기반으로 검색
2. 검색된 문서들은 잠재 변수로 취급 : 여러 문서를 확률적 후보군으로 간주함
3. 확률 기반 모델로 통합 (검색기와 생성기를 확률적으로 결합한 구조)
4. top-K 근사 + 생성

- (1) RAG-Sequence : 전체 답변이 **하나의 문서**를 기반으로 생성됨
- (2) RAG-Token : **각 단어 토큰마다 다른 문서**를 참고하여 생성 가능

5. End-to-end 학습 : 검색기와 생성기는 end-to-end로 함께 학습 및 파인튜닝 가능

• RAG는 모든 Seq2seq 작업에 파인튜닝 가능

<기존 모델들과 비교>

기존 모델들 : 외부 메모리, 즉 여러 문서 중에서 정답에 유효한 문서를 구별해서 선택하고 활용하는 작업을 위해서는 "어떤 문서가 중요한지"를 판단해야 함. 다시 말해서, **외부 메모리**

를 구별적으로 접근할 수 있는 모델 구조가 필요한데, 기존 모델인 BERT나 GPT 등의 모델들은 이를 지원하지 않아 구조를 처음부터 새로 설계할 수밖에 없었음

- 작동 원리
 1. 검색기 구조 만들기
 2. 그걸 학습시키기 (문서 유사도 학습)
 3. 생성기 구조 만들기
 4. 그걸 학습시키기 (문장 생성, 요약 등)
 5. 검색기와 생성기를 연결하고 또 학습
- 태스크가 바뀔 때마다 구조도 처음부터 다시 만들어야 함

RAG의 구조

RAG는 이미 사전학습된 모델들을 활용해서, 새로 구조를 만들거나 처음부터 학습할 필요 없이 지금 있는 것들을 조합해서 곧바로 지식을 활용할 수 있음

구조

- **검색기 (DPR)** : 이미 위키피디아 기반으로 학습된 모델
- **생성기 (BART)** : 이미 방대한 텍스트로 사전학습된 모델
- 검색기와 생성기를 수학적으로 연결 ($P(y|x) = \sum_z P(y|x,z)P(z|x)$) + 필요하면 약간의 파인튜닝

2. Method

2.1 Models

RAG-Sequence Model : 질문 입력 x 에 대해 관련된 답변 y 를 하나의 문서를 바탕으로 전체 문장을 생성

1. 검색기(DPR)를 통해 **top-K 문서 검색** (cosine similarity로 근접한 top-K 문서 선택)
2. 각각의 문서는 하나의 잠재 변수로 간주

3. 생성기(BART)를 통해 각 문서에 대한 출력 시퀀스 확률 계산 : $p(y|x, z_k)$
4. 각 문서가 관련 있을 확률 $p(z_k|x)$ 을 계산해서 두 확률을 곱하고, 모든 문서에 대해 더함 (**주변화**)

*주변화 : 중간 매개인 z (문서)중 어떤 z 가 맞는지 모르니까, z 의 모든 후보에 대해 더해서 평균을 내는 것

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y|x, z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) \prod_i^N p_{\theta}(y_i|x, z, y_{1:i-1})$$

RAG-Token Model : 질문 입력 x 에 대해 관련된 답변 y 를 각 토큰마다 다른 문서를 바탕으로 생성

- RAG-Sequence Model 과정과 같게 흘러가되, 위의 과정을 **토큰마다 반복**함

$$p_{\text{RAG-Token}}(y|x) \approx \prod_i^N \sum_{z \in \text{top-}k(p(\cdot|x))} p_{\eta}(z|x) p_{\theta}(y_i|x, z, y_{1:i-1})$$

2.2 Retriever : DPR

DPR : Bi-Encoder 구조

- $d(z) = \text{BERT}_d(z)$: 문서 인코더로 생성된 **문서의 밀집 벡터 표현**
- $q(x) = \text{BERT}_q(x)$: 쿼리 인코더로 생성된 **쿼리의 표현**
- 검색 확률 : x 에 대해 문서 z 가 **정답 생성에 도움이 될 가능성**을 확률적으로 나타낸 것
-> $d(z)$ 와 $q(x)$ 의 내적의 지수화한 값에 비례
- top-K 문서 검색 방식 : 최대 내적 검색 (MIPS)
 - MIPS : 각 문서 z 를 벡터로 표현한 후 쿼리 x 벡터와 내적 값을 계산해서 가장 큰 값을 가지는 top-k 문서를 찾는 작업

2.3 Generator : BART

생성기 모델 : 4억 개의 파라미터를 가진 사전학습된 Seq2seq Transformer인 BART-large

- 입력값
 - x와 z의 단순 연결 값
- 노이즈에 강건하도록 사전학습됨
- 성능
 - 다양한 생성 작업에서 SOTA 성능 달성

*파라메트릭 메모리 : BART 생성기의 파라미터 θ

2.4 Training

- 학습 방향 : 정답에 대한 negative marginal log-likelihood 최소화
- 최적화 : Adam optimizer를 사용한 SGD
- 파인튜닝 : 문서 인코더와 색인은 고정하고, 쿼리 인코더와 BART만 파인튜닝
(∵ 문서 색인을 갱신하는 작업은 비용이 크고, 반드시 필요하다고 판단되지는 않음)

2.5 Decoding

RAG-Token

- **Beam Search** : 토큰을 하나씩 생성하면서, 가능성 높은 후보들을 여러 개 유지하며 문장을 만들어가는 디코딩 방식

-> 각 토큰마다 확률을 알면 쓸 수 있음

-> RAG-Token은 각 토큰에 대해 평균을 낸 확률을 구할 수 있고, 이로 인해 Beam Search에 잘 맞는 구조 !!

RAG-Sequence

RAG-Sequence는 토큰별 확률이 아니라, **전체 문장에 대한 확률만 존재함**

- 문제 : Beam Search는 단어 하나씩 생성하면서 확률을 누적해야 하는데, RAG-Sequence는 문장 전체가 있어야만 확률을 계산할 수 있음 !!!!
 - 방법 1. **Thorough Decoding**

1. 각 문서 z 에 대해 별도로 Beam Search 수행해 y 에 대한 후보 문장 세트 생성
 2. 모든 문서 z 에 대해 나온 문장 후보들에 대해 각각의 확률을 z 별로 곱하고 평균냄 (주변화)
 3. 가장 확률 높은 문장 y 선택
- 방법 2. **Fast Decoding** (방법 1의 문제 : 정밀하지만 너무 느림)
 1. Beam Search 중 생성되지 않은 y 에 대해 확률을 0으로 가정하고 무시
 2. Beam Search로 생성된 y 만 가지고 확률 평균냄
 3. 가장 확률 높은 문장 y 선택

3. Experiments

- 문서 데이터 : 2018년 12월 기준 위키피디아 덤프 사용 (각 문서를 100단어 단위로 분할, 총 약 2,100만 개 문서 생성)
- 문서 색인화 : FAISS와 HNSW를 사용하여 색인화
- 문서 검색 방식 : Top-K (학습 : 5 or 10 / 평가 : 적절한 k 설정) 개의 관련 문서를 검색하여 사용

3.1 Open-domain Question Answering

Open-domain Question Answering : 모든 주제에 대해 외부 문서를 참고하거나 생성 모델을 통해 자연어로 답을 만들어내는 과제

- 학습 방향 : negative log-likelihood 최소화
- 비교 대상
 1. 추출형 QA : 문서에서 정답이 포함된 구간을 추출하는 방식 (검색 O, 생성 X)
 2. 폐쇄형 QA : 검색을 사용하지 않고, RAG처럼 답변을 생성하는 방식 (검색 X, 생성 O)
- 사용 데이터셋
 - Natural Questions (NQ)

- TriviaQA (TQA) : TQA-Wiki 테스트 세트도 평가에 포함 (∵ T5와 공정한 비교를 위해)
- WebQuestions (WQ) : NQ로 학습된 RAG 모델로 초기화 (∵ 소규모이기 때문)
- CuratedTrec (CT) : NQ로 학습된 RAG 모델로 초기화
- 데이터 분할 : 기존 연구와 동일하게 train/dev/test split 사용
- 성능 지표 : Exact Match (EM) 사용

3.2 Abstractive Question Answering

Abstractive Question Answering : 질문에 대해 자유형식의 문장 생성으로 답변하는 과제

- 비교 대상
 - MSMARCO 기반 모델들
 - 일부 질문은 gold 문단 없이는 정답 생성이 어려움
 - 일부 질문은 위키피디아만으로는 답변이 불가능함
 - RAG는 파라메트릭 메모리에 의존하여 답변 생성함
- 사용 데이터셋
 - MSMARCO NLG v2.1
 - 질문
 - 각 질문에 대해 검색 엔진에서 검색된 10개의 gold 문단
 - 해당 문단에서 주석된 완전한 문장 형태의 정답
- 실험
 - gold 문단은 사용하지 않고, 오직 질문과 정답만 활용

3.3 Jeopardy Question Generation

Jeopardy Question Generation : 정답이 주어지고, 해당 정답에 대한 사실 기반 문장으로 질문을 생성하는 과제

- 목적 : RAG의 자연어 생성 능력 평가
- 사용 데이터셋
 - SearchQA (Train : 100K, Dev : 14K, Test : 27K)
- 비교 대상
 - BART 모델
- 평가 방법
 - 자동 평가 : Q-BLEU-1
 - 엔티티(중요 정보)가 얼마나 잘 포함했는지에 더 큰 가중치 부여
 - 사람 평가
 - 사실성 : 신뢰할 수 있는 외부 자료로 뒷받침할 수 있는지
 - 구체성 : 입력과 출력 사이의 상호 의존도가 높은지
 - 평가 방식 : 쌍 비교
 - BART에서 생성한 질문 하나, RAG에서 생성한 질문 하나 제시
-> 질문 A가 낫다 / 질문 B가 낫다 / 둘 다 좋다 / 둘 다 좋지 않다 中 1 선택

3.4 Fact Verification

FEVER : 자연어로 된 주장이 위키피디아에 의해 지지되는지, 반박되는지, 아니면 판단하기에 정보가 부족한지를 분류하는 과제이자, 위키피디아 문서를 바탕으로 주장이 사실인지, 거짓인지 혹은 위키피디아만으로는 판단 불가인지를 추론하는 과제

- 목적 : RAG의 분류 능력 평가
- 실험
 - 3분류 과제 : 지지 / 반박 / 정보 부족
 - 2분류 과제 : 지지 / 반박

- RAG는 기존 모델들과 달리, 검색된 문서에 대한 감독 없이 학습함
-> 더 일반적인 적용 가능성 !!
- 성능 지표 : Label Accuracy

4. Results

4.1 Open-domain Question Answering

↑ RAG가 거의 모든 벤치마크에서 SOTA

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9/ -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

↓
T5와 비교가능한 데이터 분할에서만 !!

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] *Uses gold context/evidence. Best model without gold access underlined.

Model	Jeopardy B-1	QB-1	MSMARCO R-L	B-1	FVR3 Label	FVR2 Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

↳ BART 대비 생성 성능, 사실 정확도 일관되게 우수함.

- 문서를 재정렬하거나 특정 문장을 추출하지 않아도 SoTA
- RAG는 고비용 사전학습 없이도 우수한 성능
- 정답이 없어도 추론 가능

4.2 Abstractive Question Answering

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] *Uses gold context/evidence. Best model without gold access underlined.

Model	Jeopardy B-1	Jeopardy QB-1	MSMARCO R-L	FEVER3 B-1	FEVER3 Label	FEVER2 Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	89.5
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

주목할 만한 조건

- 기존의 다른 모델들은 gold 문단에 접근할 수 있는 상태에서 학습 및 생성 수행함
- 많은 질문들이 gold 문단이 주어지지 않으면 정답 생성이 사실상 불가능함
- 일부 질문들은 Wikipedia만으로는 절대 정답을 유도할 수 없음

-> 그럼에도 불구하고 RAG가 높은 성능을 보임

Table 3: Examples from generation tasks. RAG models generate more specific and factually accurate responses. ‘?’ indicates factually incorrect responses, * indicates partially correct responses.

Task	Input	Model	Generation
MS-MARCO	define middle ear	BART	?The middle ear is the part of the ear between the middle ear and the nose.
		RAG-T	The middle ear is the portion of the ear internal to the eardrum.
		RAG-S	The middle ear includes the tympanic cavity and the three ossicles.
	what currency needed in scotland	BART	The currency needed in Scotland is Pound sterling.
		RAG-T	Pound is the currency needed in Scotland.
		RAG-S	The currency needed in Scotland is the pound sterling.
Jeopardy Question Generation	Washington	BART	?This state has the largest number of counties in the U.S.
		RAG-T	It's the only U.S. state named for a U.S. president
		RAG-S	It's the state where you'll find Mount Rainier National Park
	The Divine Comedy	BART	"This epic poem by Dante is divided into 3 parts: the Inferno, the Purgatorio & the Purgatorio
		RAG-T	Dante's "Inferno" is the first part of this epic poem
		RAG-S	This 14th century work is divided into 3 sections: "Inferno", "Purgatorio" & "Paradiso"

- RAG가 BART보다 사실적으로 정확한 텍스트를 더 자주 생성함
- RAG가 BART보다 더 다양한 문장을 생성함

4.3 Jeopardy Question Generation

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] *Uses gold context/evidence. Best model without gold access underlined.

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	40.1	41.5	72.5	<u>89.5</u>
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

RAG-Token > RAG-Sequence

Table 4: Human assessments for the Jeopardy Question Generation Task.

	사실성 Factuality	구체성 Specificity
BART better	7.1%	16.8%
RAG better	42.7%	37.4%
Both good	11.7%	11.8%
Both poor	17.7%	6.9%
No majority	20.8%	20.1%

인간평가 : RAG > BART !!

Table 3: Examples from generation tasks. RAG models generate more specific and factually accurate responses. '?' indicates factually incorrect responses, * indicates partially correct responses.

Task	Input	Model	Generation
MS-MARCO	define middle ear	BART	?The middle ear is the part of the ear between the middle ear and the nose.
		RAG-T	The middle ear is the portion of the ear internal to the eardrum.
		RAG-S	The middle ear includes the tympanic cavity and the three ossicles.
	what currency needed in scotland	BART	The currency needed in Scotland is Pound sterling.
		RAG-T	Pound is the currency needed in Scotland.
		RAG-S	The currency needed in Scotland is the pound sterling.
Jeopardy Question Generation	Washington	BART	?This state has the largest number of counties in the U.S.
		RAG-T	It's the only U.S. state named for a U.S. president
		RAG-S	It's the state where you'll find Mount Rainier National Park
	The Divine Comedy	BART	*This epic poem by Dante is divided into 3 parts: the Inferno, the Purgatorio & the Purgatorio
		RAG-T	Dante's "Inferno" is the first part of this epic poem
		RAG-S	This 14th century work is divided into 3 sections: "Inferno", "Purgatorio" & "Paradiso"

RAG-Token: 여러 문서 결합 → 가장 좋은 정보를 뽑아 주 있음 !!

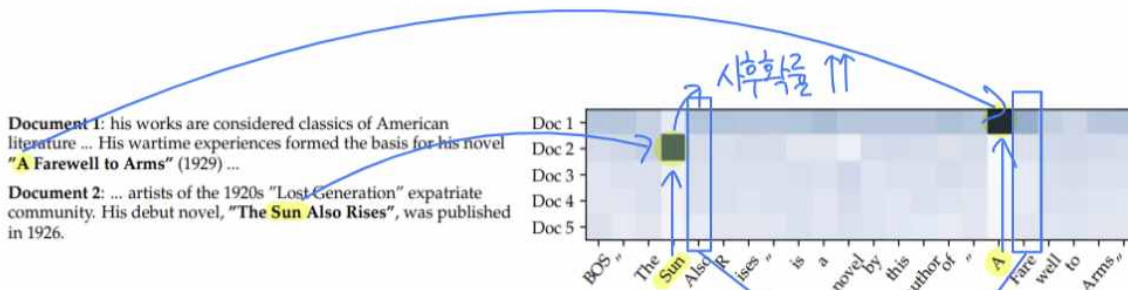


Figure 2: RAG-Token document posterior $p(z_i|x, y_i, y_{-i})$ for each generated token for input “Hemingway” for Jeopardy generation with 5 retrieved documents. The posterior for document 1 is high when generating “A Farewell to Arms” and for document 2 when generating “The Sun Also Rises”.

첫 번째 토큰 생성 후,
 → 문서별 시퀀스 확률 분포 평탄 -
 ↳ 특정 문서에 의존 X!!

But, BART는 특정 문서에 의존하지 않고 내재적인 지식만으로 내용을 완성할 수 없음 !!
 ex. "The Sun Also Rises" is a novel by this author of "The Sun Also Rises"
 ex. "The Sun Also Rises" is a novel by this author of "A Farewell to Arms"

4.4 Fact Verification

Table 1: Open-Domain QA Test Scores. For TQA, left column uses the standard test set for Open-Domain QA, right column uses the TQA-Wiki test set. See Appendix D for further details.

	Model	NQ	TQA	WQ	CT
Closed Book	T5-11B [52]	34.5	- /50.1	37.4	-
	T5-11B+SSM[52]	36.6	- /60.5	44.7	-
Open Book	REALM [20]	40.4	- / -	40.7	46.8
	DPR [26]	41.5	57.9 / -	41.1	50.6
	RAG-Token	44.1	55.2/66.1	45.5	50.0
	RAG-Seq.	44.5	56.8/ 68.0	45.2	52.2

Table 2: Generation and classification Test Scores. MS-MARCO SotA is [4], FEVER-3 is [68] and FEVER-2 is [57] *Uses gold context/evidence. Best model without gold access underlined.

Model	Jeopardy		MSMARCO		FVR3	FVR2
	B-1	QB-1	R-L	B-1	Label	Acc.
SotA	-	-	49.8*	49.9*	76.8	92.2*
BART	15.1	19.7	38.2	41.6	64.0	81.1
RAG-Tok.	17.3	22.2	<u>40.1</u>	41.5	72.5	89.5
RAG-Seq.	14.7	21.4	<u>40.8</u>	<u>44.2</u>		

첫째: SotA 모델들과 4.3% 이내 차이
→ 큰 차이 없는 성능을 냄!
(supervision 없음이)

둘째: 2.7% 이내 차이
→ gold 문장 사용 없이 비슷한 수준의
정확도를 냄!

- 가장 먼저 검색된 문서가 정답 문서일 확률 : 71%
-> RAG는 대부분 정답 문서를 1순위로 정확히 찾아냄
- 상위 10개 검색 결과 내 정답 문서 포함 비율 : 90%
-> 거의 항상 정답 문서가 상위 검색 결과에 포함됨

4.5 Additional Results

Generation Diversity

→ 서로 다른 모델이 생성한 전체 n-그램 대비
고유한 n-그램의 비율 계산 → 생성 다양성

Table 5: Ratio of distinct to total tri-grams for generation tasks.

	MSMARCO	Jeopardy QGen
Gold	89.6%	90.0%
BART	70.7%	32.4%
RAG-Token	77.8%	46.8%
RAG-Seq.	83.5%	53.8%

→ RAG-Token보다는 RAG-Seq가! ←

→ RAG가 BART보다 훨씬 더 다양한 출력 !!

Retrieval Ablations

→ Retrieval 방식에 따라
RAG의 성능이 어떻게 달라지는지 비교

Table 6: Ablations on the dev set. As FEVER is a classification task, both RAG models are equivalent.

Model	NQ	TQA Exact Match	WQ	CT	Jeopardy-QGen B-1	MSMarco R-L	FVR-3 Label	FVR-2 Accuracy
RAG-Token-BM25	29.7	41.5	32.1	33.1	17.5	55.5	75.1	91.6
RAG-Sequence-BM25	31.8	44.1	36.6	33.8	11.1	56.5	75.1	91.6
RAG-Token-Frozen	37.8	50.1	37.1	51.1	16.7	55.9	72.9	89.4
RAG-Sequence-Frozen	41.2	52.1	41.8	52.6	11.8	56.7	72.9	89.4
RAG-Token	43.5	54.8	46.5	51.9	17.9	56.2	74.5	90.6
RAG-Sequence	44.0	55.8	44.9	53.4	15.3	57.2	74.5	90.6

- BM25 : 전통적인 단어 기반 2중 Retrieval
- Frozen : DPR기반 Retrieval, 하위 2중
- 기보 : 하위 가능한 DPR 사용

→ 하위 가능한 상태로 두는 것이 모든 주요 태스크에서 성능 ↑

(But, FEVER 태스크에서는 BM25가 가장 좋은 성능을 보임)

∴ FEVER의 주상 문장이 인티티 중심 → 단어 중복 기반 검색 방식과 잘 맞음)

Index hot-swapping

논파라메트릭 메모리 모델의 장점 : 테스트 시점에서도 지식을 쉽게 업데이트할 수 있음

실험

- 사용한 색인

- 2016년 12월 위키피디아 덤프 기반의 색인
- 2018년 12월 위키피디아 덤프 기반의 색인
- 질문
 - 독립 변수 - 질문의 대상 (2016년과 2018년 사이 바뀐 82명의 세계 지도자)
 - 통제 변수 - 질문 형식 ("Who is {}?")
- 결과
 - 2016년 질문 - 2016년 색인 : 70% 정답률
 - 2018년 질문 - 2018년 색인 : 68% 정답률
 - 2016년 질문 - 2018년 색인 : 12% 정답률
 - 2018년 질문 - 2016년 색인 : 4% 정답률

RAG는 단순히 논파라메트릭 메모리를 교체함으로써 업데이트할 수 있음 !

Effect of Retrieving more documents

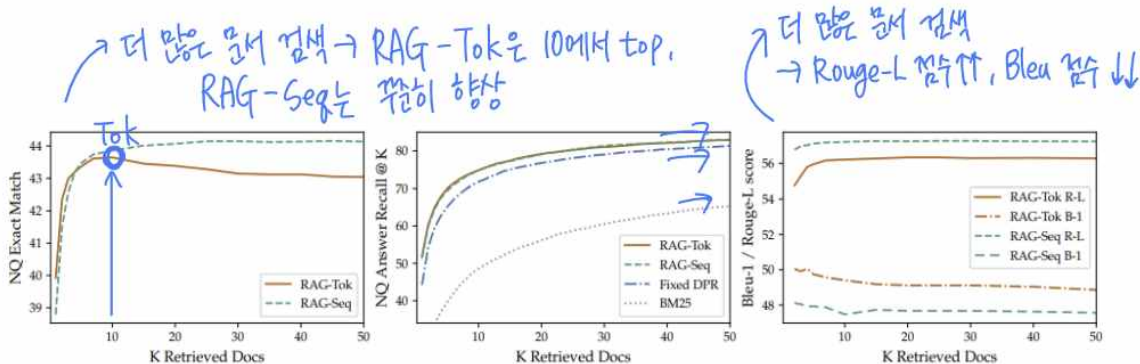


Figure 3: Left: NQ performance as more documents are retrieved. Center: Retrieval recall performance in NQ. Right: MS-MARCO Bleu-1 and Rouge-L as more documents are retrieved.

↳ 테스트 샘플에서 검색 문서의 개수의 영향 (학답은 5 or 10)

5. Related Work

Single-Task Retrieval

이전 연구들 : 검색이 개별적으로 고려된 다양한 NLP 과제에서 성능이 향상함을 보임

RAG : 개별 과제에 검색을 통합한 기존의 성공 사례들을 통합해, 하나의 검색기 기반 아키텍처로도 여러 과제에서 높은 성능을 보임

General-Purpose Architectures for NLP

이전 연구들 : 검색을 사용하지 않고, 사전학습된 모델을 파인튜닝하는 방식으로 높은 성능을 보임

RAG : 사전학습된 생성 언어 모델을 보강하는 검색 모듈을 학습함으로써, 단일 통합 아키텍처로 더 많은 과제에 적용 가능함을 보임

Learned Retrieval

이전 연구들 : 서로 다른 검색 기반 아키텍처와 최적화 기법을 활용하여 단일 작업에서 높은 성능을 달성함

RAG : 하나의 검색 기반 아키텍처만으로도 여러 작업에 대해 높은 성능을 낼 수 있도록 파인튜닝할 수 있음을 보임

Memory-based Architectures

이전 연구들 : 문서 인덱스를 분산표현으로 사용하며, 학습된 임베딩을 기반으로 대화 텍스트를 생성함 / TF-IDF 방식으로 검색함

RAG : 원시텍스트를 기반으로 함으로써, 사람이 읽을 수 있고, 편집 가능함 / end-to-end로 학습된 검색기를 사용함

Retrieve-and-Edit approaches

이전 연구들 : 유사한 입력-출력 쌍을 검색하여, 이를 편집해 최종 출력을 생성함

RAG : 검색된 항목을 가볍게 편집하기보다는, 다수의 검색 결과를 종합하여 생성에 활용함

6. Discussion

- 파라메트릭 모델과 논파라메트릭 메모리를 결합한 **하이브리드 메모리 구조** 제안함
- **학습 없이**도 검색 인덱스를 교체할 수 있음
- 생성기와 검색기를 처음부터 함께 학습하는 미래 연구 방향 제안

Broader Impact

Positive

- 실제 지식 기반을 활용해 **환각 현상이 적고, 더 사실적이고 해석 가능성 높은 결과**를 생성함
- 다양한 분야에 적용하거나 오픈 도메일 질문 대응 등 사회적 활용 가능성이 높음

Negative

- 외부 지식은 편향되거나 부정확할 수 있다는 **지식 출처의 한계**가 있음
- **악용 가능성**이 존재함 (허위·조작 정보 생성, 타인 사칭, 스팸·피싱 자동화)